

Automação de Resumo Diário

Calendar · Gmail · Drive

Registro completo da conversa, decisões tomadas e script de implementação.

Documento gerado por Claude · 27 de agosto de 2026

1. Contexto e Objetivo
2. Alternativas de Automação Apresentadas
3. Automação Escolhida
4. Decisões Definidas pelo Usuário
5. Teste Executado na Conversa
6. Modo de Acionamento
7. Script Python — Implementação
8. Script JavaScript (Node.js) — Implementação

O ponto de partida

O usuário solicitou sugestões de automação envolvendo três conectores disponíveis: Google Calendar, Gmail e Google Drive. A partir dessas sugestões, uma delas foi escolhida para detalhamento e implementação: um resumo diário de compromissos com os anexos de e-mail correspondentes organizados automaticamente no Drive.

Alternativas de Automação Apresentadas

ALTERNATIVA	DESCRIÇÃO
Resumo diário de compromissos com anexos organizados	Todo dia, verifica eventos do Calendar, busca e-mails relacionados no Gmail e salva anexos no Drive em pasta por data.
Criação automática de eventos a partir de e-mails	Identifica convites/datas em e-mails recebidos, cria o evento no Calendar e anexa documentos do Drive.
Arquivo pós-reunião automatizado	Após o término de um evento, consolida documentos, e-mails trocados e decisões em um arquivo único no Drive.

Automação Escolhida

Resumo diário de compromissos com anexos organizados. Fluxo conceitual definido na conversa:

- 1 Consultar o Google Calendar e identificar os compromissos do dia.
- 2 Para cada compromisso, buscar no Gmail e-mails relacionados.
- 3 Baixar os anexos relevantes encontrados nesses e-mails.
- 4 Salvar os anexos no Google Drive, organizados em uma pasta nomeada por data.
- 5 Apresentar um resumo final ao usuário, com os compromissos do dia e os documentos já organizados.

Decisões Definidas pelo Usuário

PARÂMETRO	DECISÃO
Relação e-mail ↔ compromisso	Por assunto do e-mail (nome da reunião)
Organização dos anexos no Drive	Uma pasta por dia, nomeada com a data (ex: 27-08-2026)
Modo de acionamento	Manual, sob demanda — sem agendamento automático nesta plataforma

Teste Executado na Conversa

Antes de finalizar a lógica, foi executada uma consulta real ao Google Calendar do usuário para o dia 27/08/2026, como prova de conceito do fluxo.

Resultado: nenhum compromisso foi encontrado na agenda para essa data — portanto, não havia e-mails ou anexos a processar naquele momento.

Modo de Acionamento

Foi esclarecido ao usuário que este ambiente não possui um recurso nativo de agendamento (cron/scheduler) para execução autônoma e recorrente. Assim, ficou definido que a automação será acionada manualmente, bastando o usuário enviar uma mensagem como "roda meu resumo diário" a qualquer momento, preferencialmente pela manhã.

Script Python — Implementação

Abaixo, a lógica da automação convertida para um script Python de referência, utilizando as APIs oficiais do Google (Calendar, Gmail e Drive). O script segue fielmente as decisões definidas: correspondência por assunto do e-mail, pasta diária no Drive e execução sob demanda.

```
resumo_diario_automacao.py
```

```
"""
```

```
Automação de Resumo Diário de Compromissos
```

```
=====
```

```
Fluxo:
```

1. Busca os compromissos do dia no Google Calendar.
2. Para cada compromisso, busca no Gmail e-mails cujo ASSUNTO corresponda ao nome/título da reunião.
3. Baixa os anexos relevantes e salva no Google Drive, em uma pasta nomeada com a data do dia (ex: 27-08-2026).
4. Gera um resumo final com os compromissos e os anexos organizados.

```
Regras de negócio definidas pelo usuário:
```

- Relacionamento e-mail <-> compromisso: por ASSUNTO do e-mail.
- Organização no Drive: uma pasta por dia (DD-MM-AAAA).
- Disparo: manual, sob demanda (ex: "roda meu resumo diário"), sem agendamento automático.

```
Requisitos:
```

```
pip install --break-system-packages google-api-python-client \
    google-auth google-auth-oauthlib
```

```
Autenticação:
```

```
Este script assume que já existe um fluxo OAuth2 configurado  
(arquivo token.json / credentials.json) com os escopos:
```

- <https://www.googleapis.com/auth/calendar.readonly>
- <https://www.googleapis.com/auth/gmail.readonly>
- <https://www.googleapis.com/auth/drive.file>

```
"""
```

```
from __future__ import annotations
```

```
import base64
```

```
import datetime as dt
```

```
import os
```

```
from dataclasses import dataclass, field
```

```
from typing import List, Optional
```

```
from google.oauth2.credentials import Credentials
```

```
from googleapiclient.discovery import build
```

```
from googleapiclient.http import MediaIoBaseUpload
```

```
import io
```

```
SCOPES = [
```

```
    "https://www.googleapis.com/auth/calendar.readonly",
```

```
    "https://www.googleapis.com/auth/gmail.readonly",
```

```
    "https://www.googleapis.com/auth/drive.file",
```

```
]
```

```

TOKEN_PATH = "token.json"

# ----- #
# Modelos de dados
# ----- #

@dataclass
class Compromisso:
    id: str
    titulo: str
    inicio: dt.datetime
    fim: dt.datetime
    participantes: List[str] = field(default_factory=list)

@dataclass
class Anexo:
    nome_arquivo: str
    conteudo: bytes
    mime_type: str
    email_assunto: str

# ----- #
# Autenticação
# ----- #

def carregar_credenciais() -> Credentials:
    """Carrega as credenciais OAuth2 já autorizadas (token.json)."""
    if not os.path.exists(TOKEN_PATH):
        raise FileNotFoundError(
            "token.json não encontrado. Rode o fluxo de autorização OAuth2 "
            "antes de usar este script."
        )
    return Credentials.from_authorized_user_file(TOKEN_PATH, SCOPES)

# ----- #
# Etapa 1: Google Calendar – buscar compromissos do dia
# ----- #

def buscar_compromissos_do_dia(
    creds: Credentials, data: Optional[dt.date] = None
) -> List[Compromisso]:
    """Retorna todos os compromissos do dia informado (padrão: hoje)."""
    data = data or dt.date.today()
    service = build("calendar", "v3", credentials=creds)

    inicio_dia = dt.datetime.combine(data, dt.time.min).isoformat() + "Z"
    fim_dia = dt.datetime.combine(data, dt.time.max).isoformat() + "Z"

    resultado = (
        service.events()
        .list(
            calendarId="primary",
            timeMin=inicio_dia,
            timeMax=fim_dia,
            singleEvents=True,
            orderBy="startTime",
        )
    )

```

```

        .execute()
    )

    compromissos = []
    for evento in resultado.get("items", []):
        inicio = evento["start"].get("dateTime", evento["start"].get("date"))
        fim = evento["end"].get("dateTime", evento["end"].get("date"))
        participantes = [
            p.get("email") for p in evento.get("attendees", []) if p.get("email")
        ]
        compromissos.append(
            Compromisso(
                id=evento["id"],
                titulo=evento.get("summary", "(sem título)"),
                inicio=dt.datetime.fromisoformat(inicio),
                fim=dt.datetime.fromisoformat(fim),
                participantes=participantes,
            )
        )
    return compromissos

# ----- #
# Etapa 2: Gmail – localizar e-mails cujo assunto bate com o compromisso
# ----- #

def buscar_emails_por_assunto(creds: Credentials, titulo_reuniao: str) -> List[dict]:
    """Busca e-mails no Gmail cujo assunto contenha o título da reunião."""
    service = build("gmail", "v1", credentials=creds)
    query = f'subject:"{titulo_reuniao}" has:attachment'

    resultado = service.users().messages().list(userId="me", q=query).execute()
    mensagens = resultado.get("messages", [])

    detalhes = []
    for msg in mensagens:
        detalhe = (
            service.users().messages().get(userId="me", id=msg["id"]).execute()
        )
        detalhes.append(detalhe)
    return detalhes

def extrair_anexos(creds: Credentials, mensagem: dict) -> List[Anexo]:
    """Extrai os anexos de uma mensagem do Gmail."""
    service = build("gmail", "v1", credentials=creds)
    anexos = []

    assunto = next(
        (h["value"] for h in mensagem["payload"]["headers"] if h["name"] == "Subject"),
        "(sem assunto)",
    )

    partes = mensagem.get("payload", {}).get("parts", []) or []
    for parte in partes:
        if parte.get("filename") and parte.get("body", {}).get("attachmentId"):
            anexo_id = parte["body"]["attachmentId"]
            anexo_dados = (
                service.users()
                .messages()

```

```

        .attachments()
        .get(userId="me", messageId=mensagem["id"], id=anexo_id)
        .execute()
    )
    conteudo = base64.urlsafe_b64decode(anexo_dados["data"])
    anexos.append(
        Anexo(
            nome_arquivo=parte["filename"],
            conteudo=conteudo,
            mime_type=parte.get("mimeType", "application/octet-stream"),
            email_assunto=assunto,
        )
    )
    return anexos

# ----- #
# Etapa 3: Google Drive – organizar anexos em pasta por dia
# ----- #

def obter_ou_criar_pasta_do_dia(
    creds: Credentials, data: Optional[dt.date] = None
) -> str:
    """Obtém (ou cria) a pasta do Drive nomeada com a data (DD-MM-AAAA)."""
    data = data or dt.date.today()
    nome_pasta = data.strftime("%d-%m-%Y")
    service = build("drive", "v3", credentials=creds)

    query = (
        f"name = '{nome_pasta}' and mimeType = 'application/vnd.google-apps.folder' "
        "and trashed = false"
    )
    resultado = service.files().list(q=query, fields="files(id, name)").execute()
    pastas = resultado.get("files", [])

    if pastas:
        return pastas[0]["id"]

    metadata = {
        "name": nome_pasta,
        "mimeType": "application/vnd.google-apps.folder",
    }
    pasta = service.files().create(body=metadata, fields="id").execute()
    return pasta["id"]

def salvar_anexo_no_drive(creds: Credentials, pasta_id: str, anexo: Anexo) -> str:
    """Salva um anexo dentro da pasta do dia no Drive."""
    service = build("drive", "v3", credentials=creds)

    media = MediaIoBaseUpload(
        io.BytesIO(anexo.conteudo), mimetype=anexo.mime_type, resumable=True
    )
    metadata = {"name": anexo.nome_arquivo, "parents": [pasta_id]}

    arquivo = (
        service.files().create(body=metadata, media_body=media, fields="id").execute()
    )
    return arquivo["id"]

```

```

# ----- #
# Etapa 4: Orquestração – gerar o resumo diário completo
# ----- #

def rodar_resumo_diario(data: Optional[dt.date] = None) -> str:
    """Executa o fluxo completo e retorna um resumo em texto."""
    creds = carregar_credenciais()
    data = data or dt.date.today()

    compromissos = buscar_compromissos_do_dia(creds, data)
    if not compromissos:
        return f"Nenhum compromisso encontrado para {data.strftime('%d/%m/%Y')}."

    pasta_id = obter_ou_criar_pasta_do_dia(creds, data)

    linhas_resumo = [f"Resumo de {data.strftime('%d/%m/%Y')}:\n"]

    for compromisso in compromissos:
        linhas_resumo.append(
            f"- {compromisso.inicio.strftime('%H:%M')} | {compromisso.titulo}"
        )

        emails = buscar_emails_por_assunto(creds, compromisso.titulo)
        total_anexos = 0

        for email in emails:
            anexos = extrair_anexos(creds, email)
            for anexo in anexos:
                salvar_anexo_no_drive(creds, pasta_id, anexo)
                total_anexos += 1

        if total_anexos:
            linhas_resumo.append(
                f"    -> {total_anexos} anexo(s) salvo(s) na pasta do dia no Drive."
            )
        else:
            linhas_resumo.append("    -> Nenhum e-mail/anexo relacionado encontrado.")

    return "\n".join(linhas_resumo)

if __name__ == "__main__":
    print(rodar_resumo_diario())

```

Arquivo completo disponível separadamente como `resumo_diario_automacao.py`.

Script JavaScript (Node.js) — Implementação

Versão equivalente da automação em JavaScript (Node.js), usando a biblioteca oficial `googleapis`. Mantém exatamente a mesma lógica e as mesmas decisões definidas pelo usuário: correspondência por assunto do e-mail, pasta diária no Drive e execução sob demanda.

```
resumo_diario_automacao.js
```

```
/**
 * Automação de Resumo Diário de Compromissos
 * =====
 *
 * Fluxo:
 * 1. Busca os compromissos do dia no Google Calendar.
 * 2. Para cada compromisso, busca no Gmail e-mails cujo ASSUNTO
 *    corresponda ao nome/título da reunião.
 * 3. Baixa os anexos relevantes e salva no Google Drive, em uma
 *    pasta nomeada com a data do dia (ex: 27-08-2026).
 * 4. Gera um resumo final com os compromissos e os anexos organizados.
 *
 * Regras de negócio definidas pelo usuário:
 * - Relacionamento e-mail <-> compromisso: por ASSUNTO do e-mail.
 * - Organização no Drive: uma pasta por dia (DD-MM-AAAA).
 * - Disparo: manual, sob demanda (ex: "roda meu resumo diário"),
 *   sem agendamento automático.
 *
 * Requisitos:
 * npm install googleapis
 *
 * Autenticação:
 * Este script assume que já existe um fluxo OAuth2 configurado
 * (arquivo token.json / credentials.json) com os escopos:
 * - https://www.googleapis.com/auth/calendar.readonly
 * - https://www.googleapis.com/auth/gmail.readonly
 * - https://www.googleapis.com/auth/drive.file
 */

const fs = require("fs");
const { google } = require("googleapis");
const { Readable } = require("stream");

const SCOPES = [
  "https://www.googleapis.com/auth/calendar.readonly",
  "https://www.googleapis.com/auth/gmail.readonly",
  "https://www.googleapis.com/auth/drive.file",
];

const TOKEN_PATH = "token.json";

// ----- //
// Autenticação
// ----- //

/**
 * Carrega as credenciais OAuth2 já autorizadas (token.json) e devolve
 * um cliente autenticado do Google.
 */
```

```

function carregarCredenciais() {
  if (!fs.existsSync(TOKEN_PATH)) {
    throw new Error(
      "token.json não encontrado. Rode o fluxo de autorização OAuth2 " +
      "antes de usar este script."
    );
  }

  const token = JSON.parse(fs.readFileSync(TOKEN_PATH, "utf-8"));

  const oAuth2Client = new google.auth.OAuth2(
    token.client_id,
    token.client_secret,
    token.redirect_uri
  );
  oAuth2Client.setCredentials(token);

  return oAuth2Client;
}

// ----- //
// Etapa 1: Google Calendar – buscar compromissos do dia
// ----- //

/**
 * Retorna todos os compromissos do dia informado (padrão: hoje).
 * @param {import('google-auth-library').OAuth2Client} auth
 * @param {Date} [data]
 * @returns {Promise<Array<Object>>}}
 */
async function buscarCompromissosDoDia(auth, data = new Date()) {
  const calendar = google.calendar({ version: "v3", auth });

  const inicioDia = new Date(data);
  inicioDia.setHours(0, 0, 0, 0);
  const fimDia = new Date(data);
  fimDia.setHours(23, 59, 59, 999);

  const resultado = await calendar.events.list({
    calendarId: "primary",
    timeMin: inicioDia.toISOString(),
    timeMax: fimDia.toISOString(),
    singleEvents: true,
    orderBy: "startTime",
  });

  const eventos = resultado.data.items || [];

  return eventos.map((evento) => ({
    id: evento.id,
    titulo: evento.summary || "(sem título)",
    inicio: new Date(evento.start.dateTime || evento.start.date),
    fim: new Date(evento.end.dateTime || evento.end.date),
    participantes: (evento.attendees || []).
      .map((p) => p.email)
      .filter(Boolean),
  }));
}

// ----- //

```

```
// Etapa 2: Gmail – localizar e-mails cujo assunto bate com o compromisso
// ----- //

/**
 * Busca e-mails no Gmail cujo assunto contenha o título da reunião.
 */
async function buscarEmailsPorAssunto(auth, tituloReuniao) {
  const gmail = google.gmail({ version: "v1", auth });
  const query = `subject:"${tituloReuniao}" has:attachment`;

  const resultado = await gmail.users.messages.list({
    userId: "me",
    q: query,
  });

  const mensagens = resultado.data.messages || [];

  const detalhes = [];
  for (const msg of mensagens) {
    const detalhe = await gmail.users.messages.get({
      userId: "me",
      id: msg.id,
    });
    detalhes.push(detalhe.data);
  }
  return detalhes;
}

/**
 * Extraí os anexos de uma mensagem do Gmail.
 */
async function extrairAnexos(auth, mensagem) {
  const gmail = google.gmail({ version: "v1", auth });
  const anexos = [];

  const headers = mensagem.payload.headers || [];
  const assunto =
    headers.find((h) => h.name === "Subject")?.value || "(sem assunto)";

  const partes = mensagem.payload.parts || [];

  for (const parte of partes) {
    if (parte.filename && parte.body?.attachmentId) {
      const anexoDados = await gmail.users.messages.attachments.get({
        userId: "me",
        messageId: mensagem.id,
        id: parte.body.attachmentId,
      });

      const conteudo = Buffer.from(anexoDados.data.data, "base64");

      anexos.push({
        nomeArquivo: parte.filename,
        conteudo,
        mimeType: parte.mimeType || "application/octet-stream",
        emailAssunto: assunto,
      });
    }
  }
  return anexos;
}
```

```

}

// ----- //
// Etapa 3: Google Drive — organizar anexos em pasta por dia
// ----- //

/**
 * Obtém (ou cria) a pasta do Drive nomeada com a data (DD-MM-AAAA).
 */
async function obterOuCriarPastaDoDia(auth, data = new Date()) {
  const drive = google.drive({ version: "v3", auth });

  const dia = String(data.getDate()).padStart(2, "0");
  const mes = String(data.getMonth() + 1).padStart(2, "0");
  const ano = data.getFullYear();
  const nomePasta = `${dia}-${mes}-${ano}`;

  const query =
    `name = '${nomePasta}' and mimeType = 'application/vnd.google-apps.folder' ` +
    "and trashed = false";

  const resultado = await drive.files.list({
    q: query,
    fields: "files(id, name)",
  });

  const pastas = resultado.data.files || [];
  if (pastas.length > 0) {
    return pastas[0].id;
  }

  const pasta = await drive.files.create({
    requestBody: {
      name: nomePasta,
      mimeType: "application/vnd.google-apps.folder",
    },
    fields: "id",
  });

  return pasta.data.id;
}

/**
 * Salva um anexo dentro da pasta do dia no Drive.
 */
async function salvarAnexoNoDrive(auth, pastaId, anexo) {
  const drive = google.drive({ version: "v3", auth });

  const arquivo = await drive.files.create({
    requestBody: {
      name: anexo.nomeArquivo,
      parents: [pastaId],
    },
    media: {
      mimeType: anexo.mimeType,
      body: Readable.from(anexo.conteudo),
    },
    fields: "id",
  });
}

```

```

    return arquivo.data.id;
}

// ----- //
// Etapa 4: Orquestração – gerar o resumo diário completo
// ----- //

/**
 * Executa o fluxo completo e retorna um resumo em texto.
 * @param {Date} [data]
 * @returns {Promise<string>}
 */
async function rodarResumoDiario(data = new Date()) {
    const auth = carregarCredenciais();

    const compromissos = await buscarCompromissosDoDia(auth, data);
    const dataFormatada = data.toLocaleDateString("pt-BR");

    if (compromissos.length === 0) {
        return `Nenhum compromisso encontrado para ${dataFormatada}.`;
    }

    const pastaId = await obterOuCriarPastaDoDia(auth, data);

    const linhasResumo = [`Resumo de ${dataFormatada}:\n`];

    for (const compromisso of compromissos) {
        const hora = compromisso.inicio.toLocaleTimeString("pt-BR", {
            hour: "2-digit",
            minute: "2-digit",
        });
        linhasResumo.push(`- ${hora} | ${compromisso.titulo}`);

        const emails = await buscarEmailsPorAssunto(auth, compromisso.titulo);
        let totalAnexos = 0;

        for (const email of emails) {
            const anexos = await extrairAnexos(auth, email);
            for (const anexo of anexos) {
                await salvarAnexoNoDrive(auth, pastaId, anexo);
                totalAnexos += 1;
            }
        }

        if (totalAnexos > 0) {
            linhasResumo.push(
                `    -> ${totalAnexos} anexo(s) salvo(s) na pasta do dia no Drive.`
            );
        } else {
            linhasResumo.push("    -> Nenhum e-mail/anexo relacionado encontrado.");
        }
    }

    return linhasResumo.join("\n");
}

// ----- //
// Execução direta via linha de comando
// ----- //
```

```
if (require.main === module) {
  rodarResumoDiario()
    .then((resumo) => console.log(resumo))
    .catch((erro) => {
      console.error("Erro ao rodar o resumo diário:", erro.message);
      process.exit(1);
    });
}

module.exports = {
  carregarCredenciais,
  buscarCompromissosDoDia,
  buscarEmailsPorAssunto,
  extrairAnexos,
  obterOuCriarPastaDoDia,
  salvarAnexoNoDrive,
  rodarResumoDiario,
};
```

Arquivo completo disponível separadamente como `resumo_diario_automacao.js`.

FORMAÇÃO CLAUDE CODE 2026 – AUTOMAÇÃO DE RESUMO DIÁRIO (CALENDAR · GMAIL · DRIVE)